Editorial: Problem C - Jayden's T-Spin Setup

Setter: Hudson

Problem Overview

We are tasked with simulating a simplified game of Tetris. We have a 10×20 grid. Pieces (I, O, T, L, J, S, Z) are dropped one by one into specific columns without rotation. We need to determine the final height of the stack, or report "Game Over" if it exceeds height 20.

Key Challenges

1. **Grid Representation:** We need a way to store which cells are occupied. A 2D boolean array `grid[row][col]` works perfectly. Since the max height is 20, but we need to detect overflows, we can make the array slightly larger (e.g., 50 rows) to safely handle calculations above the ceiling.
2. **Shape Definitions:** Each piece type occupies a specific set of cells relative to its "anchor" (bottom-left corner). Hardcoding these shapes is the most robust way to handle them.
3. **Collision Detection:** "Dropping" a piece means finding the lowest row R such that the piece fits at row R but would collide with existing blocks (or the floor) at row $R-1$.

Solution Approach

1. **Data Structure:** Initialize a boolean grid `grid[50][12]` to false. Note that we use 1-based indexing for columns $1 \dots 10$. 2. **Shape Mapping:** Create a helper function that returns the list of occupied relative coordinates (dr, dc) for each piece type.

- Example T-Piece: $(0,0), (0,1), (0,2)$ (Bottom row) and $(1,1)$ (Top row middle).

3. **Simulation Loop:** For each piece input (Type T, Column C):

- Start trying to place the piece at a high row (e.g., Row 25).
- Check if moving the piece down by 1 row causes a collision.
- A collision happens if:
  - The new row index would be 0 (Floor).
  - Any block of the piece overlaps with a `true` cell in the grid.
- Once the lowest valid position is found, mark those cells as `true` in the grid.
- **Game Over Check:** If any part of the newly placed piece is at Row > 20, print "Game Over" and exit.

4. **Final Output:** If no overflow occurred after N pieces, scan the grid to find the highest row index that contains a `true` value.

Complexity

- **Time Complexity:** $O(N \times H)$, where N is the number of pieces and H is the simulation height. Since H is small constant (approx 25), this is effectively $O(N)$.
- **Space Complexity:** $O(H \times W)$ for the grid, which is negligible constant space.

Implementation (C++)

(See reference solution code)